Object-Oriented Programming

Mastering C++ Architectures

A 18-Page Comprehensive Technical Analysis

BLUEPRINTS • PILLARS • LOGIC

1. The OOP Paradigm

Object-Oriented Programming (OOP) is a programming style that organizes software design around **data**, or **objects**, rather than functions and logic. In C++, this allows us to mirror real-world complexities within computer code.

Why OOP?

- **Modularity:** Code is divided into independent building blocks.
- **Reusability:** Use previously written classes in new projects.
- **Maintainability:** It is easier to fix a specific object than a monolithic program.

The Primary Ecosystem

C++ bridges the gap between low-level performance and high-level abstraction through Classes and Objects.

2. Blueprints & Instances

> **Analogy:** A **Class** is an architectural drawing of a house. The **Object** is the actual house built from that drawing. One drawing (Class) can build thousands of houses (Objects).

Defining a Class

```
class CodeHive {
public: // Access Specifier
    string course;
    int duration;

    void showDetails() {
        cout << "Course: " << course;
    }
};
```

Creating Objects

```
int main() {
    CodeHive cpp; // Object instantiation
    cpp.course = "C++ Masterclass";
    cpp.showDetails();
}
```

3. Access Control

Security is a core part of OOP. Access specifiers define who can see and "touch" the internal data of your objects.

Specifier	Description
`public`	Accessible from outside the class (Public API).
`private`	Accessible **only** within the class itself.
`protected`	Accessible to the class and its inherited children.

```
class BankAccount {
private:
    double balance; // Cannot be accessed directly from main()
public:
    void deposit(double amt) { balance += amt; }
};
```

4. Constructors

A **Constructor** is a special member function that is called automatically when an object is born. It has the same name as the class and no return type.

Purpose of Constructors:

1. Setting default values.
2. Allocating memory.
3. Ensuring an object is "ready for use" upon creation.

```
class Hero {
public:
    string name;
    Hero(string n) { // Constructor
        name = n;
        cout << name << " has joined the game!";
    }
};
```

5. Custom Initialization

Constructors can take parameters, allowing you to create unique objects in a single line of code.

```
class Car {
public:
    string brand;
    int year;

    // Parameterized Constructor
    Car(string b, int y) : brand(b), year(y) {}
};

int main() {
    Car myCar("Tesla", 2024);
}
```

> **Note:** If you don't write any constructor, C++ provides a "Default" one for you. However, as soon as you write a custom one, the default disappears!

6. Destructors

The **Destructor** is the opposite of a constructor. It is called just before an object is removed from memory (e.g., when the function ends or the program finishes).

Syntax: `~ClassName()`

```
class Database {
public:
    Database() { cout << "Connected!"; }
    ~Database() { cout << "Connection Closed."; }
};
```

> Think of the Destructor as the person who turns off the lights and locks the door after everyone has left the building.

7. Pillar: Encapsulation

Encapsulation is the practice of bundling data and methods into a single unit (Class) and restricting direct access to the data.

The Getter/Setter Pattern:

```
class Employee {
private:
    int salary;
public:
    void setSalary(int s) { // Setter
        if (s > 0) salary = s;
    }
    int getSalary() { // Getter
        return salary;
    }
};
```

This allows the class to validate data before saving it, ensuring the object's state is always valid.

8. Pillar: Inheritance

Inheritance allows a new class (derived) to take on the traits of an existing class (base). This is the key to code reuse.

```
class Animal {
public:
    void eat() { cout << "Eating ... "; }
};

class Dog : public Animal { // Dog inherits Animal
public:
    void bark() { cout << "Woof!"; }
};
```

> **Relation:** Inheritance establishes an "IS-A" relationship. A Dog is-an Animal.

9. Hierarchy Patterns

C++ supports complex inheritance structures that allow for sophisticated data models.

1. Single Inheritance

One Parent → One Child

2. Multi-level Inheritance

Grandparent → Parent → Child

3. Multiple Inheritance

Parent A + Parent B → Child (C++ supports this, unlike Java!)

```
class SmartPhone : public Camera, public Phone {
    // Inherits features from BOTH
};
```

10. Pillar: Polymorphism

Polymorphism means "many forms." It allows a single action to behave differently based on the object it is acting upon.

Types of Polymorphism:

- **Compile-time:** Function overloading.
- **Run-time:** Function overriding (Virtual Functions).

```
void makeSound(Animal *a) {
    a->speak(); // Behavior changes based on Animal type
}
```

11. Run-time Polymorphism

By using the `virtual` keyword, you tell C++: "Wait until the program is running to decide which version of this function to use."

```
class Shape {
public:
    virtual void draw() { cout << "Generic Shape"; }
};

class Circle : public Shape {
public:
    void draw() override { cout << "Circle"; }
};
```

> Without `virtual`, C++ would always call the Shape version, even if the object was actually a Circle!

12. Pillar: Abstraction

Abstraction is the act of hiding complex implementation details and showing only the essential features of an object.

> **Analogy:** When you use a smartphone, you know how to tap an app (Interface), but you don't need to know how the processor handles the electricity (Implementation).

Abstract Classes

Classes with a **Pure Virtual Function** cannot be instantiated as objects. They exist only to be inherited.

13. Interface Design

A **Pure Virtual Function** is defined by setting it to `= 0`. This creates a contract: "Any child class must write their own version of this function."

```
class RemoteControl {
public:
    virtual void turnOn() = 0; // Pure Virtual
};

class TVRemote : public RemoteControl {
public:
    void turnOn() { cout << "TV Glowing!"; }
};
```

14. Objects in Memory

Objects can be created in two locations, affecting their lifespan.

Stack Objects

Automatic cleanup. Created normally: `Hero h;`

Heap Objects

Manual cleanup. Created with new: `Hero *h = new Hero();`

> **Memory Warning:** Heap objects must be deleted manually using `delete h;`, otherwise you will have a memory leak!

15. The 'this' Pointer

Every object has access to its own address through a pointer named `this`. It is useful when parameter names are the same as member variable names.

```
class Player {
    int health;
public:
    void setHealth(int health) {
        this→health = health; // Clarifies which 'health' is which
    }
};
```

16. Static: Shared Knowledge

Usually, every object has its own variables. However, a `static` variable is shared by **all** objects of that class.

```
class User {
public:
    static int userCount; // Shared by ALL users
    User() { userCount++; }
};

int User::userCount = 0; // Initialize outside
```

This is perfect for tracking global data related to a class, such as counting how many players are currently in a game room.

17. Summary & Best Practices

- **Class:** The Blueprint.
- **Object:** The realized Instance.
- **Encapsulation:** Data Secrecy (Getters/Setters).
- **Inheritance:** Reusing Code (Base/Derived).
- **Polymorphism:** Dynamic Behavior (Virtual).
- **Abstraction:** Simplifying Information (Pure Virtual).

Final Pro-Tip:

Think in terms of "What is this thing?" (Class) and "What can this thing do?" (Methods) before you write any code. OOP is about design first, syntax second.

MASTERED: C++ OBJECT-ORIENTED PROGRAMMING